

CSCE 753 COMPUTER VISION · MODULE 08 · LECTURE NOTES

Learned Representations

What a network learns about images, and what to reuse.

MODULE OUTCOMES

- Explain what convolutional features represent at each depth.
- Explain transfer learning and choose a transfer strategy.
- Explain receptive field and why it constrains architecture.
- Explain the role of resolution in vision models.
- State what learned features cannot do.

1 What the layers learn

Depth	What the filters respond to	Comment
Layer 1	Oriented edges and colour-opponent blobs.	Gabor-like, essentially always, in every architecture and under almost any training objective. See the callout.
Layers 2–3	Corners, junctions, simple textures.	Recognisably the features of Module 03 — the Harris corner response has a clear analogue here.
Mid depth	Object parts — wheels, eyes, text-like patterns, characteristic textures.	The most transferable layers , and the part that was never successfully hand-designed.
Late layers	Object-like and increasingly category-specific.	Tied to the training task, so less transferable the further the target task is from the source.
Final layer	The training classes themselves.	Discard on transfer ; it encodes the source label set and nothing more.

LEARNED EARLY LAYERS REDISCOVERED HAND-DESIGNED ONES

The first convolutional layer reliably learns oriented edge detectors and colour-opponent filters — which is what Gabor filters are, and what SIFT's gradient-orientation histograms were built out of.

Nobody put them there. They emerge from gradient descent on natural images under classification, under self-supervision (Module 10), and under reconstruction objectives alike. **The hierarchy is a property of natural image statistics, not of the architecture or the task.**

So the hand-designed features of Module 03 were not arbitrary. Decades of design converged on approximately the right primitives, **which is exactly why learned descriptors beat SIFT only modestly** (Module 03 §4) — a result that is puzzling until this module and obvious after it.

What learning added was the middle of the hierarchy. Parts and their configurations were never successfully hand-designed — the attempts (deformable part models, constellation models) were ingenious and did not scale — **and that is where the gains in recognition came from.** Knowing which part of the stack learning actually contributed is more useful than the general claim that it works.

2 Transfer

UNDER 1000 LABELLED IMAGES

FREEZE the backbone and train a linear head only.
Anything more overfits. A linear probe on good features is a strong baseline and frequently enough.

1k - 10k IMAGES

Fine-tune the **LAST FEW BLOCKS**, freeze the rest. Use a learning rate about 10x lower than you would train from scratch -- the features are already good and large steps destroy them before the head can adapt.

10k - 100k IMAGES

Fine-tune **EVERYTHING** with a layer-wise decaying learning rate: lowest at layer 1, highest at the head. Early layers need almost no change (section 1).

OVER 100k IMAGES

Fine-tuning still beats training from scratch, but the margin narrows. Here pretraining is mostly buying optimisation speed rather than final accuracy.

ALWAYS: compare against the **FROZEN LINEAR PROBE**. If fine-tuning does not beat it, you are overfitting and reporting it as progress. That is CSCE 633 Module 03's lesson in the exact form it takes in vision, and the probe costs one line of code.

3 Receptive field and resolution

A UNIT CANNOT RESPOND TO ANYTHING OUTSIDE ITS RECEPTIVE FIELD

The receptive field is the region of the input image that can influence a given unit's activation, and it grows with depth, with stride, and with dilation — computable by a short recurrence over the layers.

If the object you need to recognise spans 200 pixels and the receptive field at your decision layer is 80 pixels, no single unit can see the whole object, and no amount of training or data fixes it. The information is not available to that unit.

So compute it rather than assuming it. A mismatch between object scale and receptive field is a common and completely invisible architectural error — the model trains, the loss decreases, and the ceiling is structural.

And the *effective* receptive field is considerably smaller than the theoretical one: contributions fall off roughly as a Gaussian from the centre, because there are far more paths through the network to the centre than to the periphery. **So build in margin rather than matching the theoretical figure exactly**, and prefer architectures with explicit global context (pooling, attention) when the task needs it.

- **Small objects occupy few pixels**, and after four stride-2 downsamplings a 32-pixel object occupies 2 pixels in the feature map. **It is effectively gone**, and the detector's small-object performance reflects exactly that.
- **So detection across scales needs features at multiple resolutions** — which is what feature pyramid networks exist for, and **it is Module 03's scale pyramid again**, arrived at independently and for the same reason.
- **Compute scales quadratically with input resolution**, so **resolution is the single most expensive architectural choice** and deserves to be chosen from the smallest object that must be detected rather than by convention.
- **And the training and test resolution must match.** If you train on 224-pixel crops of 256-pixel images and test on full 512-pixel images, **the apparent object scale shifts and accuracy drops for no visible reason** — which is a **distribution shift** (Module 12) **that you introduced yourself, in the data pipeline, with no model defect at all.** It is a frequent bug and it presents as a mysterious accuracy gap rather than as an error.

4 What learned features cannot do

STATED PLAINLY

They do not give metric geometry. A network can estimate relative depth within a scene quite well and absolute scale not at all — because **Module 01's scale theorem binds learned methods exactly as hard as geometric ones**. A monocular depth network that appears to output metric depth has learned the typical size of objects in its training distribution, which is a prior and fails on an unusual room.

They do not generalise outside their training distribution, and the degradation is neither graceful nor predictable — performance can fall sharply on a shift that looks minor to a person. This is Module 12's entire subject.

They do not provide usable uncertainty without explicit effort. Softmax confidence is not calibrated (CSCE 633 Module 12 §3), and a confidently wrong detection is indistinguishable from a confidently right one at the interface.

And they are less interpretable than §1's clean hierarchy suggests. The 'edges, then parts, then objects' story is a partial truth assembled from visualisations that select for interpretability — the units that produce legible visualisations are a minority, many units are polysemantic, and the story is a useful organising picture rather than a description of the whole network. **Hold it loosely, and do not use it to justify a claim about why a particular prediction was made.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Michigan EECS 498 — CNN architectures and transfer lectures (free)	The reference for this module. The transfer-learning lecture covers §2 directly.
Zeiler & Fergus — Visualizing and Understanding Convolutional Networks (free)	Where §1's hierarchy picture comes from , and worth reading alongside §4's caveat about it.
Araujo, Norris & Sim — Computing Receptive Fields of Convolutional Neural Networks (free, Distill)	The §3 recurrence, worked out , with interactive calculators.
Touvron et al. — Fixing the train-test resolution discrepancy (free)	The §3 bug, measured and fixed — a good example of a data-pipeline problem mistaken for a model problem.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Visualise the first-layer filters** of three pretrained models and compare them.
- 2 **Compare them against Gabor filters** you generate yourself.
- 3 **Train a linear probe** on frozen features for a small task and record the accuracy.
- 4 **Fine-tune the whole network on the same data** and compare. Report which wins and at what dataset size the crossover occurs.
- 5 **Compute the receptive field** of each layer of a network you use, by hand.
- 6 **Verify it empirically** by occluding input regions and measuring which units change.
- 7 **Train at 224 and test at 384** and report the accuracy change.
- 8 **Correct for it** and report the recovered accuracy.
- 9 **Take a monocular depth model** and test it on a scene with unusually sized objects. Report the scale error.
- 10 **Find three units in a mid-layer** and try to characterise what each responds to. Report how many were interpretable.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is learned at each depth, and which layers transfer best?
- 2 Why is it striking that layer 1 learns Gabor filters?
- 3 What did learning add that hand design never achieved?
- 4 Give the transfer strategy for each data regime.
- 5 Why is the linear probe the essential baseline?
- 6 What is a receptive field, and why must it be computed?
- 7 Why is resolution the most expensive choice?
- 8 Explain the train/test resolution bug.
- 9 Name four things learned features cannot do.